

编 号	题 号	标 题	知 识 点	代 码 作 者
例 7-1	UVa 10305	Ordering Tasks	拓扑排序	陈锋
例 7-2	POJ 1386	Play on Words	欧拉回路	刘汝佳
例 7-3	UVa 12118	Inspector's Dilemma	欧拉道路	陈锋
例 7-4	SPOJ KNIGHTS	Knights of the Round Table	点-双连通分量；二分图判定	刘汝佳
例 7-5	SPOJ BUSINESS	Mining Your Own Business	点-双连通分量；割点	陈锋
例 7-6	HDU 2767	Proving Equivalences	SCC； SCC 图	刘汝佳
例 7-7	UVa 11324	The Largest Clique	SCC； DAG 动态规划	陈锋
例 7-8	UVa 1391	Astronauts	2-SAT	刘汝佳

7.2 最短路问题

形形色色的最短路问题恰恰是图论中最优美、最吸引人的问题。本节介绍Floyd算法、BellmanFord算法以及Dijkstra算法相关的经典例题以及代码实现。

例 7-9 【Floyd 算法；传递闭包】电话圈（Calling Circles, ACM/ICPC World Finals 1996, UVa 247）

如果两个人相互打电话（直接或间接），则说他们在同一个电话圈里。例如，a打给b，b打给c，c打给d，d打给a，则这4个人在同一个电话圈里；如果e打给f但f不打给e，则不能推出e和f在同一个电话圈里。输入 n ($n \leq 25$) 个人的 m 次电话，找出所有电话圈。人名只包含字母，不超过25个字符，且不重复。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;
const int NN = 32;
int N, M, G[NN][NN], VIS[NN];
vector<string> names;
map<string, int> indice;
int getID(const string& s) { // 获取结点 ID
    if (!indice.count(s)) indice[s] = names.size(), names.push_back(s);
    return indice[s];
}

int main() {
    string A, B;
    for (int t = 1; cin >> N >> M && N; t++) {
        if (t > 1) cout << endl;
        names.clear(), indice.clear();
        memset(G, 0, sizeof(G)), fill_n(VIS, N + 1, false);
        _for(i, 0, M) cin >> A >> B, G[getID(A)][getID(B)] = 1;
        assert(names.size() <= N);
        _for(k, 0, N) _for(i, 0, N) _for(j, 0, N)
            G[i][j] = G[i][j] || (G[i][k] && G[k][j]);
        cout << "Calling circles for data set " << t << ":" << endl;
        _for(u, 0, N) {
            if (VIS[u]) continue;
            cout << names[u], VIS[u] = 1;
            _for(v, 0, N) if (G[u][v] && G[v][u] && !VIS[v])
                VIS[v] = 1, cout << ", " << names[v];
            cout << endl;
        }
    }

    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 11-4
同类题目：Rank, HDU 1704
*/

```

例7-10 【Floyd算法；最大值最小路】噪声恐惧症（Audiophobia, UVa 10048）

输入一个 C 个点、 S 条边（ $C \leq 100$ ， $S \leq 1000$ ）的无向带权图，边权表示该路径上的噪声值。当噪声值太大时，耳膜可能会受到伤害，所以当你从某点去往另一个点时，总是希望经过的路上的最大噪声值最小。输入一些询问，每次询问两个点，输出这两点间最大噪声值最小的路径。例如，在图7-1中， A 到 G 的最大噪声值为80，是所有其他路径中最小的（如 $A \rightarrow B \rightarrow E \rightarrow G$ 的最大噪声值为90）。

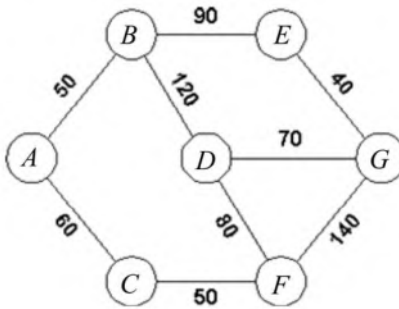


图7-1 路径与噪声值

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;
const int MAXC = 100 + 4, INF = 1e9;
int G[MAXC][MAXC];

int main() {
    for (int kase = 1, C, S, Q, c1, c2, d;
         scanf("%d%d%d", &C, &S, &Q) == 3 && C; kase++) {
        if (kase > 1) puts("");
        printf("Case #%d\n", kase);
        _for(i, 0, C) _for(j, 0, C) G[i][j] = i == j ? 0 : INF;
        _for(i, 0, S) {
            scanf("%d%d%d", &c1, &c2, &d), --c1, --c2;
            G[c1][c2] = min(G[c1][c2], d), G[c2][c1] = G[c1][c2];
        }
        _for(k, 0, C) _for(i, 0, C) _for(j, 0, C) {
            if (G[i][k] != INF && G[k][j] != INF)
                G[i][j] = min(G[i][j], max(G[i][k], G[k][j]));
        }
        _for(i, 0, Q) {
            scanf("%d%d", &c1, &c2), --c1, --c2;
            int ans = G[c1][c2];
            if (ans == INF) puts("no path");
            else printf("%d\n", ans);
        }
    }

    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 11-5
注意：本题在 floyd 算法实现中，为安全起见，更新 G[i][j] 之前判定了相关元素不为 INF
同类题目：Say Cheese, ACM/ICPC WF 2001, UVa 1001
*/

```

例7-11 【用Dijkstra预处理；枚举】机场快线（Airport Express, UVa 11374）

在Iokh市中，机场快线是市民从市内去机场的首选交通工具。机场快线分为经济线和商业线两种，线路、速度和价钱都不同。你有一

张商业线车票，可以乘坐一站商业线，而其他时候只能乘坐经济线。假设换乘时间忽略不计，你的任务是找出一条去机场最快的线路。

输入包含多组数据。每组数据：第一行为3个整数 N , S 和 E ($2 \leq N \leq 500$, $1 \leq S, E \leq 100$)，即机场快线中的车站总数、起点编号和终点（即机场所在站）编号；下一行包含一个整数 M ($1 \leq M \leq 1000$)，即经济线的路段条数；以下 M 行，每行3个整数 X , Y 和 Z ($1 \leq X, Y \leq N$, $1 \leq Z \leq 100$)，表示可以乘坐经济线在车站 X 和车站 Y 之间往返，其中单程需要 Z 分钟；下一行为商业线的路段条数 K ($1 \leq K \leq 1000$)；以下 K 行是这些路段的描述，格式同经济线。所有路段都是双向的，但有可能必须使用商业车票才能到达机场。输入保证最优解唯一。

对于每组数据，输出3行。第一行按访问顺序给出经过的各个车站（包括起点和终点），第二行是换乘商业线的车站编号（如果没用商业线车票，输出 Ticket Not Used），第三行是总时间。

【代码实现】

```
// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
```

```

typedef long long LL;
struct Edge {
    int u, v, d;
    bool operator<(const Edge& e) const { return d < e.d; }
};

struct HeapNode {
    int u, d;
    bool operator<(const HeapNode& rhs) const { return d > rhs.d; }
};

template <int SZV, int INF>                // |V|
struct Dijkstra {
    int n;
    vector<Edge> edges;
    vector<int> G[SZV];
    bool done[SZV];
    int d[SZV], p[SZV];

    void init(int n) {
        assert(n < SZV);
        this->n = n, edges.clear();
        _for(i, 0, n) G[i].clear();
    }

    void addEdge(int u, int v, int d) { // u-v,d
        G[u].push_back(edges.size()), edges.push_back({u, v, d});
    }

    void dijkstra(int s) {
        priority_queue<HeapNode> Q;
        fill_n(done, n, false), fill_n(d, n, INF);
        d[s] = 0, Q.push({s, 0});
        while (!Q.empty()) {
            HeapNode x = Q.top();
            Q.pop();
            int u = x.u;                // 距离已经确定最短路的点集最近的点
            if (done[u]) continue;
            done[u] = true;
            for (size_t ei = 0; ei < G[u].size(); ei++) {
                const auto& e = edges[G[u][ei]];
                int v = e.v;
                if (d[v] > d[u] + e.d)
                    d[v] = d[u] + e.d, p[v] = G[u][ei], Q.push({v, d[v]});
            }
        }
    }

    // 获得 s->e 的路径

```

```

void getPath(int s, int e, deque<int>& path, bool rev = false) {
    assert(d[s] == 0), assert(d[e] != INF);
    int x = e;
    if (rev) path.push_back(x);
    else path.push_front(x);
    while (x != s) {
        x = edges[p[x]].u;
        if (rev) path.push_back(x);
        else path.push_front(x);
    }
}

};

const int MAXN = 500 + 4, INF = 1e9;
int main() {
    Dijkstra<MAXN, INF> SD, ED; // S -> * , E -> *
    for (int t = 0, N, S, E, M, K, u, v, d; scanf("%d%d%d", &N, &S, &E) == 3; t++) {
        if (t) puts("");
        SD.init(N + 1), ED.init(N + 1);
        scanf("%d", &M);
        for (int i = 0; i < M; i++) { // 经济快线
            scanf("%d%d%d", &u, &v, &d);
            SD.addEdge(u, v, d), SD.addEdge(v, u, d);
            ED.addEdge(u, v, d), ED.addEdge(v, u, d);
        }
        SD.dijkstra(S), ED.dijkstra(E);
        int cu = -1, ans = INF;
        deque<int> path;
        if (SD.d[E] < ans) ans = SD.d[E], SD.getPath(S, E, path);
        // lambda实现局部函数
        auto update = [&](int u, int v, int d) { // S -> u -> v -> E;
            if (SD.d[u] < ans && ED.d[v] < ans && SD.d[u] + d + ED.d[v] < ans) {
                ans = SD.d[u] + d + ED.d[v], cu = u, path.clear();
                SD.getPath(S, u, path), ED.getPath(E, v, path, true);
            }
        };

        scanf("%d", &K);
        _for(i, 0, K)
            scanf("%d%d%d", &u, &v, &d), update(u, v, d), update(v, u, d);
        _for(i, 0, path.size()) {
            if (i) printf(" ");
            printf("%d", path[i]);
        }
        puts("");
        if (cu == -1) puts("Ticket Not Used");
        else printf("%d\n", cu);
        printf("%d\n", ans);
    }
}

```

```

    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.3.1 节例题 11
注意：本题使用了 C++11 中提供的 lambda 来实现一些子过程的封装
同类题目：Walk Through the Forest, UVa 10917
*/

```

例7-12 【最短路树】战争和物流 (Warfare And Logistics, UVa 1416)

给出一个包含 n 个结点、 m 条边的无向图 ($1 < n \leq 100$, $1 \leq m \leq 1000$)，每条边上有一个正权。令 c 等于每对结点的最短路长度之和。例如， $n=3$ 时， $c = d(1,1) + d(1,2) + d(1,3) + d(2,1) + d(2,2) + d(2,3) + d(3,1) + d(3,2) + d(3,3)$ 。要求删除一条边后，使得新的 c 值 c' 最大。不连通的两点的最短路长度视为 L ($1 \leq L \leq 10^8$)。输出 c 的原始值和删除一条边后的最大值 c' 。

【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <cstring>
#include <vector>
#include <algorithm>
#include <queue>
using namespace std;

const int INF = 1e9, NN = 100 + 8;
struct Edge { int from, to, dist; };
struct HeapNode {
    int d, u;
    bool operator < (const HeapNode& rhs) const {
        return d > rhs.d;
    }
};

template<size_t SZ>
struct Dijkstra {
    int n, m;
    vector<Edge> edges;
    vector<int> G[SZ];
    bool done[SZ]; // 是否已永久标号
    int d[SZ]; // s 到各个点的距离
    int p[SZ]; // 最短路中的上一条弧

    void init(int n) {
        this->n = n;
        for (int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, int dist) {

```

```

    edges.push_back((Edge) {from, to, dist});
    m = edges.size(), G[from].push_back(m - 1);
}

void dijkstra(int s) {
    priority_queue<HeapNode> Q;
    fill_n(d, n + 1, INF), fill_n(done, n + 1, false);
    d[s] = 0, Q.push((HeapNode) {0, s});
    while (!Q.empty()) {
        HeapNode x = Q.top(); Q.pop();
        int u = x.u;
        if (done[u]) continue;
        done[u] = true;
        for (size_t i = 0; i < G[u].size(); i++) {
            Edge& e = edges[G[u][i]]; // 此处和模板不同，忽略了 dist = -1 的边
            if (e.dist > 0 && d[e.to] > d[u] + e.dist) {
                // 此为删除标记，根据题意和 Dijkstra 算法的前提，正常的边 dist > 0
                d[e.to] = d[u] + e.dist, p[e.to] = G[u][i];
                Q.push((HeapNode) {d[e.to], e.to});
            }
        }
    }
}

// 题目相关
Dijkstra<NN> solver;
int N, M, L;
vector<int> gr[NN][NN]; // 两点之间的原始边权
int used[NN][NN][NN]; // used[src][a][b] 表示源点为 src 的最短路树是否包含边 a->b
int idx[NN][NN]; // idx[u][v] 为边 u->v 在 Dijkstra 求解器中的编号
int sum_single[NN]; // sum_single[src] 表示源点为 src 的最短路树的所有 d 之和

int compute_c() {
    int ans = 0;
    memset(used, 0, sizeof(used));
    for (int src = 0; src < N; src++) {
        solver.dijkstra(src);
        sum_single[src] = 0;
        for (int i = 0; i < N; i++) {
            if (i != src) {
                int fa = solver.edges[solver.p[i]].from;
                used[src][fa][i] = used[src][i][fa] = 1;
            }
            sum_single[src] += (solver.d[i] == INF ? L : solver.d[i]);
        }
        ans += sum_single[src];
    }
    return ans;
}

```

```

}

int compute_newc(int a, int b) {
    int ans = 0;
    for (int src = 0; src < N; src++)
        if (!used[src][a][b]) ans += sum_single[src];
    else {
        solver.dijkstra(src);
        for (int i = 0; i < N; i++)
            ans += (solver.d[i] == INF ? L : solver.d[i]);
    }
    return ans;
}

int main() {
    while (scanf("%d%d%d", &N, &M, &L) == 3) {
        solver.init(N);
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++) gr[i][j].clear();

        for (int i = 0, a, b, s; i < M; i++) {
            scanf("%d%d%d", &a, &b, &s), a--, b--;
            gr[a][b].push_back(s), gr[b][a].push_back(s);
        }

        // 构造网络
        for (int i = 0; i < N; i++)
            for (int j = i + 1; j < N; j++) if (!gr[i][j].empty()) {
                sort(gr[i][j].begin(), gr[i][j].end());
                solver.AddEdge(i, j, gr[i][j][0]);
                idx[i][j] = solver.m - 1;
                solver.AddEdge(j, i, gr[i][j][0]);
                idx[j][i] = solver.m - 1;
            }

        int c = compute_c(), c2 = -1;
        for (int i = 0; i < N; i++) for (int j = i + 1; j < N; j++)
            if (!gr[i][j].empty()) {
                int& e1 = solver.edges[idx[i][j]].dist;
                int& e2 = solver.edges[idx[j][i]].dist;
                if (gr[i][j].size() == 1) e1 = e2 = -1;
                else e1 = e2 = gr[i][j][1]; // 第一、第二短的边
                c2 = max(c2, compute_newc(i, j));
                e1 = e2 = gr[i][j][0]; // 恢复
            }

        printf("%d %d\n", c, c2);
    }
    return 0;
}

```

```

}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.3.1 节例题 13
*/

```

例 7-13 【Dijkstra 算法的变形】过路费（加强版）（Toll! Revisited, UVa 10537）

运送货物需要缴纳过路费。假设进入一个村庄需要缴纳1个单位的货物，而进入一个城镇时，每20个单位的货物要上缴1个单位的货物（比如，携带70个单位的货物进入城镇，需要缴纳4个单位）。如图7-2所示，你必须途径一个城镇和两个村庄才能到达目的地，为了运送66把勺子，你必须携带76把勺子出发。

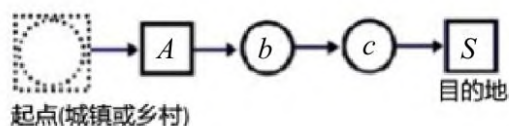


图7-2 过路费

当起点与终点固定时，到达目的地的路线往往不唯一。如图7-3所示，运送39把勺子的最佳路线是 $A \rightarrow b \rightarrow c \rightarrow X$ ，而运送10把勺子的最佳路线是 $A \rightarrow D \rightarrow X$ 。这里，大写字母表示城镇，小写字母表示村庄。

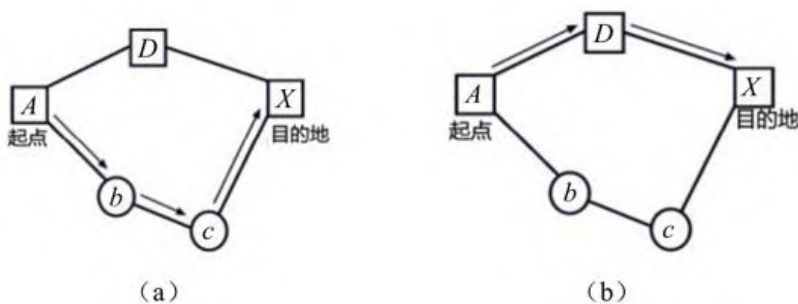


图7-3 最佳路线

你的任务是找出一条缴纳过路费最少的路线。

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cstring>
#include <algorithm>
#include <cctype>
using namespace std;

const int NN = 52 + 10;
const long long INF = 1LL << 60;
typedef long long LL;

int N, G[NN][NN], St, Ed, P, Vis[NN];      // 标记
LL D[NN];
    // D[i]表示从点 i 出发（已经交过点 i 的税）时至少要带多少东西，到 Ed 时还能剩 p 个东西
```

```

int read_node() {
    char s[9];
    scanf("%s", s);
    if (isupper(s[0])) return s[0] - 'A';
    return s[0] - 'a' + 26;
}

char node_label(int u) { return u < 26 ? 'A' + u : 'a' + (u - 26); }

LL forward(LL k, int u) { // 拿着 k 个东西去结点 u, 还剩多少个东西
    if (u < 26) return k - (k + 19) / 20; // 城镇
    return k - 1; // 村庄
}

// 至少要拿着多少个东西到达结点 u, 交税以后还能剩 D[u] 个东西
LL back(int u) {
    if (u >= 26) return D[u] + 1; // 村庄
    LL X = D[u] * 20 / 19; // 初始值
    while (forward(X, u) < D[u]) X++; // 调整, 容易理解的做法
    return X;
}

void solve() {
    N = 52; // 总是有 52 个结点
    fill_n(Vis, N + 1, 0), fill_n(D, N, INF);
    D[Ed] = P, Vis[Ed] = 1;
    for (int i = 0; i < N; i++)
        if (i != Ed && G[i][Ed]) D[i] = back(Ed);

    while (!Vis[St]) { // Dijkstra 主过程, 逆推, 规模小则不需要优先级队列
        int minu = -1; // 找最小的 D[u] 的 u
        for (int i = 0; i < N; i++)
            if (!Vis[i] && (minu < 0 || D[i] < D[minu])) minu = i;
        Vis[minu] = 1;
        for (int i = 0; i < N; i++)
            if (!Vis[i] && G[i][minu]) D[i] = min(D[i], back(minu)); // 更新其他结点的 d
    }
    printf("%lld\n%c", D[St], node_label(St)); // 输出
    LL k = D[St]; // 当前手里有多少货
    for (int u = St, next; u != Ed; u = next) {
        for (next = 0; next < N; next++) // 找到第一个可以走的结点
            if (G[u][next] && forward(k, next) >= D[next]) break;
        k = D[next];
        printf("-%c", node_label(next));
        u = next;
    }
    puts("");
}

int main() {
    for (int kase = 1; scanf("%d", &N) == 1 && N >= 0; kase++) {
        memset(G, 0, sizeof(G));
        for (int i = 0; i < N; i++) {

```

```

    int u = read_node(), v = read_node();
    if (u != v) G[u][v] = G[v][u] = 1;
}
scanf("%d", &P);
St = read_node(), Ed = read_node();
printf("Case %d:\n", kase);
solve();
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.3.1 节例题 14
*/

```

例7-14 【复杂状态的最短路】这不是bug，而是特性 (It' s not a Bug, it' s a Feature, UVa 658)

补丁在修正bug时，有时也会引入新的bug。假定有 n ($n \leq 20$) 个潜在bug和 m ($m \leq 100$) 个补丁，每个补丁用两个长度为 n 的字符串表示，其中字符串的每个位置表示一个bug。第一个串表示打补丁之前的状态（“-”表示该bug必须不存在，“+”表示必须存在，0表示无所谓），第二个串表示打补丁之后的状态（“-”表示不存在，“+”表示存在，0表示不变）。每个补丁都有一个执行时间，你的任务是用最少的时间把一个有 n 个潜在bug的软件通过打补丁的方式变得没有bug。一个补丁可以打多次。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;
const int MAXN = 20, MAXV = (1 << MAXN) + 4, INF = 0x7f7f7f7f;
struct HeapNode {
    int d, u;
    HeapNode(int _d, int _u) : d(_d), u(_u) {}
    bool operator<(const HeapNode& rhs) const { return d > rhs.d; }
};

int N, M, D[MAXV];
bitset<MAXV> Done;
struct Patch {
    int present, absent, introduce, remove, time;
    bool canApply(int u) const {
        return (present & u) == present && (absent & u) == 0;
    }
    int apply(int u) const {
        return (u | introduce) & (~remove) & ((1 << N) - 1);
    }
}

```

```

};
vector<Patch> patches;

void toInt(const char* s, int& i1, int& i2) {
    i1 = 0, i2 = 0;
    _for(i, 0, N) {
        if (s[i] == '+') i1 |= (1 << i);
        if (s[i] == '-') i2 |= (1 << i);
    }
    // assert(0 <= i1 && i1 < (1 << N));
    // assert(0 <= i2 && i2 < (1 << N));
}

int solve() {
    int V = 1 << N, st = V - 1;
    priority_queue<HeapNode> Q;
    Done.reset();
    fill_n(D, V, INF);
    D[st] = 0, Q.push(HeapNode(0, st));
    while (!Q.empty()) {
        HeapNode nd = Q.top();
        Q.pop();
        int u = nd.u;
        if (Done[u]) continue;
        Done.set(u);
        for (size_t pi = 0; pi < patches.size(); pi++) {
            const Patch& p = patches[pi];
            if (!p.canApply(u)) continue;
            int v = p.apply(u);
            if (D[v] > D[u] + p.time)
                D[v] = D[u] + p.time, Q.push(HeapNode(D[u] + p.time, v));
        }
    }

    return D[0];
}

int main() {
    char buf1[MAXN + 4], buf2[MAXN + 4];
    for (int t = 1; scanf("%d%d", &N, &M) == 2 && N && M; t++) {
        patches.clear();
        Patch p;
        _for(i, 0, M) {
            scanf("%d%s%s", &(p.time), buf1, buf2);
            toInt(buf1, p.present, p.absent), toInt(buf2, p.introduce, p.remove);
            patches.push_back(p);
        }
        int ans = solve();
        printf("Product %d\n", t);
    }
}

```

```

    if (ans == INF)
        puts("Bugs cannot be fixed.");
    else
        printf("Fastest sequence takes %d seconds.\n", ans);
        puts("");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题11-6
同类题目：Lift Hopping, UVa 10801
*/

```

例7-15 【状态图设计；Dijkstra】蒸汽式压路机（Steam Roller, UVa 1078)

有一个由 R 条横线和 C 条竖线组成的网格，你的任务是开着一辆蒸汽式压路机，用最短的时间从起始点 (r_1, c_1) 出发，最后停到目的地 (r_2, c_2) 。其中，一些线段上有权值，代表全速通过需要的时间。权值为0的边不能通过（ $1 \leq r_1, r_2 \leq R \leq 100$, $1 \leq c_1, c_2 \leq C \leq 100$ ）。

由于蒸汽式压路机的惯性较大，对于任意一条边，如果在进入这条边之前刚转弯，或者离开这条边以后立即需要转弯，则实际时间为理想值的两倍。同样的道理，通过整条路线的起点和终点所在的边也需要两倍时间。注意，“时间加倍”规则不可叠加。比如，若离开整条路线的第一条边之后立即转弯，则通过第一条边所花的时间仍是理想值的两倍而不是4倍。

沿着理想值为9的边走，总时间为 $18+18+18+18+18+18=108$ ，而沿着理想值为10的边走，总时间为 $20+10+20+20+10+20=100$ ，如图7-4所示。

开始

10 9	10	10 10
9 9		10
	9 9	10 9

结束

图7-4 蒸汽式压路机

输出最短时间。如果无法到达，则输出 “Impossible” 。

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cstring>
#include <iostream>
#include <queue>
using namespace std;
const int INF = 1e9, maxn = 50000 + 10;
struct Edge {
    int from, to, dist;
};
struct HeapNode {
    int d, u;
    bool operator<(const HeapNode& rhs) const { return d > rhs.d; }
};
struct Dijkstra {
```

```

int n, m;
vector<Edge> edges;
vector<int> G[maxn];
bool done[maxn];           // 是否已永久标号
int d[maxn];               // s 到各个点的距离
int p[maxn];               // 最短路中的上一条弧

void init(int n) {
    this->n = n;
    for (int i = 0; i < n; i++) G[i].clear();
    edges.clear();
}

void AddEdge(int from, int to, int dist) {
    edges.push_back((Edge) {from, to, dist});
    m = edges.size(), G[from].push_back(m - 1);
}

void dijkstra(int s) {
    priority_queue<HeapNode> Q;
    for (int i = 0; i < n; i++) d[i] = INF;
    d[s] = 0, fill_n(done, n + 1, false);
    Q.push((HeapNode) {0, s});
    while (!Q.empty()) {
        HeapNode x = Q.top();
        Q.pop();
        int u = x.u;
        if (done[u]) continue;
        done[u] = true;
        for (size_t i = 0; i < G[u].size(); i++) {
            Edge& e = edges[G[u][i]];
            if (d[e.to] > d[u] + e.dist) {
                d[e.to] = d[u] + e.dist, p[e.to] = G[u][i];
                Q.push((HeapNode) {d[e.to], e.to});
            }
        }
    }
}
};

```

//////// 题目相关

```
const int UP = 0, LEFT = 1, DOWN = 2, RIGHT = 3;
```

```
const int inv[] = {2, 3, 0, 1};
```

```
const int dr[] = {-1, 0, 1, 0};
```

```
const int dc[] = {0, -1, 0, 1};
```

```
const int maxr = 100, maxc = 100;
```

```
int grid[maxr][maxc][4], n, id[maxr][maxc][5], R, C;
```

// 上左下右

```

int ID(int r, int c, int dir) {
    int& x = id[r][c][dir];
    if (x == 0) x = ++n; // 从1开始编号
    return x;
}

bool cango(int r, int c, int dir) {
    if (r < 0 || r >= R || c < 0 || c >= C) return false; // 走出网格
    return grid[r][c][dir] > 0; // 此路不通
}

Dijkstra solver;

int main() {
    int r1, c1, r2, c2, kase = 0;
    while (cin >> R >> C >> r1 >> c1 >> r2 >> c2 && R) {
        r1--, c1--, r2--, c2--;
        for (int r = 0; r < R; r++) {
            for (int c = 0; c < C - 1; c++) {
                cin >> grid[r][c + 1][LEFT];
                grid[r][c][RIGHT] = grid[r][c + 1][LEFT];
            }
            if (r != R - 1)
                for (int c = 0; c < C; c++) {
                    cin >> grid[r + 1][c][UP];
                    grid[r][c][DOWN] = grid[r + 1][c][UP];
                }
        }
        solver.init(R * C * 5 + 1);
        n = 0, memset(id, 0, sizeof(id));
        // 源点出发的边
        for (int dir = 0; dir < 4; dir++) {
            if (!cango(r1, c1, dir)) continue;
            solver.AddEdge(0, ID(r1 + dr[dir], c1 + dc[dir], dir),
                grid[r1][c1][dir] * 2); // 开始走下去
            solver.AddEdge(0, ID(r1 + dr[dir], c1 + dc[dir], 4),
                grid[r1][c1][dir] * 2); // 走一步停下来
        }

        // 计算每个状态(r,c,dir)的后继状态
        for (int r = 0; r < R; r++)
            for (int c = 0; c < C; c++) {
                for (int dir = 0; dir < 4; dir++) {
                    if (!cango(r, c, inv[dir])) continue;
                    solver.AddEdge(ID(r, c, dir), ID(r, c, 4),
                        grid[r][c][inv[dir]]); // 停下来
                    if (cango(r, c, dir)) // 继续走
                        solver.AddEdge(ID(r, c, dir), ID(r + dr[dir], c + dc[dir], dir),
                            grid[r][c][dir]);
                }
            }
    }
}

```

```

        for (int dir = 0; dir < 4; dir++) {
            if (!cango(r, c, dir)) continue;
            int nr = r + dr[dir], nc = c + dc[dir];
            solver.AddEdge(ID(r, c, 4), ID(nr, nc, dir), grid[r][c][dir]*2); // 重新开始
            solver.AddEdge(ID(r, c, 4), ID(nr, nc, 4), grid[r][c][dir]*2); // 走一步停下
        }
    }

    solver.dijkstra(0);
    int ans = solver.d[ID(r2, c2, 4)]; // 找最优解
    printf("Case %d: ", ++kase);
    if (ans == INF) puts("Impossible");
    else printf("%d\n", ans);
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.3.3 节例题 17
注意：本题最重要的是各种状态之间的转换
同类题目：Low Cost Air Travel, UVa 1048
*/

```

例7-16 【特殊图的Dijkstra算法】有趣的赛车比赛（Funny Car Racing, UVa 12661）

在一个赛车比赛中，赛道有 n ($n \leq 300$) 个交叉点和 m ($m \leq 50000$) 条单向道路。有趣的是，每条路都是周期性关闭的。每条路用5个整数 u, v, a, b, t 表示 ($1 \leq u, v \leq n, 1 \leq a, b, t \leq 10^5$)，表示起点的是 u ，终点的是 v ，通过时间为 t （单位为秒）。另外，这条路会打开 a 秒，然后关闭 b 秒，然后再打开 a 秒，依此类推。比赛开始时，每条道路刚刚打开。你的赛车必须在道路打开时进入该道路，并且在它关闭之前离开（进出道路不花时间，所以可以在打开的瞬间进入，关闭的瞬间离开）。

你的任务是从 S 出发，尽早到达目的地 T ($1 \leq S, T \leq n$)。道路的起点和终点不会相同，但是可能有两条道路的起点和终点分别相同。

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
const int MAXV = 305;
typedef pair<int, int> IPair;          // D[u]: D

struct Edge {
    int from, to, open, close, w;
    Edge(int f, int t, int a, int b, int c)
        : from(f), to(t), open(a), close(b), w(c) {}
};

vector<Edge> G[MAXV];
int N, M, S, T, D[MAXV];

int arrive(int t, const Edge& e) {    // 从到达道路e起点的时间t到达道路e终点时间
    int k = t % (e.open + e.close);    // e.w <= open
    if (k + e.w <= e.open) return t + e.w;    // x--t+e.w--x+a-----x+b
    return t + e.open + e.close - k + e.w;    // x--x+a--t+e.w--x+b
}

void Dijkstra() {
    priority_queue<IPair, vector<IPair>, greater<IPair>> Q;
    fill(D, D + MAXV, INT_MAX), D[S] = 0, Q.push(make_pair(0, S));
    while (!Q.empty()) {
        IPair p = Q.top();
        Q.pop();
        int u = p.second, d = p.first;
        if (D[u] != d) continue;          // d的最短路已经算出来了
        for (size_t i = 0; i < G[u].size(); i++) {
            const Edge& e = G[u][i];
            int v = e.to, &dv = D[e.to], nd = arrive(D[u], e); // 到达e.to的时间
            if (dv > nd) dv = nd, Q.push({dv, v});
        }
    }
}

int main() {
    for (int kase = 1; scanf("%d%d%d%d", &N, &M, &S, &T) == 4; ++kase) {
        for (int i = 0; i <= N; i++) G[i].clear();
        for (int i = 0; i < M; i++) {
            int u, v, a, b, t;
            scanf("%d%d%d%d%d", &u, &v, &a, &b, &t);
            if (t > a) continue;          // 通过用时比开放时间长
            G[u].push_back(Edge(u, v, a, b, t));
        }
        Dijkstra();
        printf("Case %d: %d\n", kase, D[T]);
    }
    return 0;
}

/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题11-11
注意：本题在Dijkstra算法中边的存在性与当前的d值有关
*/

```

例7-17 【01分数规划；负圈判定】在环中（Going in Cycle!!, UVa 11090）

给定一个 n ($n \leq 50$) 个点、 m 条边的加权有向图，求平均权值最小的回路。输出最小平均值。如果无解，输出“No cycle found.”。

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cstring>
#include <queue>
```

```

using namespace std;

const int INF = 1e9, NN = 1000;

struct Edge {
    int from, to;
    double dist;
};

struct BellmanFord {
    int n, m;
    vector<Edge> edges;
    vector<int> G[NN];
    bool inq[NN];           // 是否在队列中
    double d[NN];          // s 到各个点的距离
    int p[NN];             // 最短路中的上一条弧
    int cnt[NN];           // 进队次数

    void init(int n) {
        this->n = n;
        for (int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, double dist) {
        edges.push_back((Edge) {from, to, dist});
        m = edges.size(), G[from].push_back(m - 1);
    }

    bool negativeCycle() {
        queue<int> Q;
        for (int i = 0; i < n; i++)
            d[i] = 0, inq[i] = true, Q.push(i), cnt[i] = 0;
        while (!Q.empty()) {
            int u = Q.front(); Q.pop();
            inq[u] = false;
            for (int i = 0; i < G[u].size(); i++) {
                Edge& e = edges[G[u][i]];
                if (d[e.to] > d[u] + e.dist) {
                    d[e.to] = d[u] + e.dist, p[e.to] = G[u][i];
                    if (!inq[e.to]) {
                        Q.push(e.to), inq[e.to] = true;
                        if (++cnt[e.to] > n) return true; // 进队次数大于 n, 发现环
                    }
                }
            }
        }
        return false;
    }
};

```

```

};

BellmanFord solver;
bool test(double x) { // 注意，每次判负环之前要修改对应边权并且还原
    for (int i = 0; i < solver.m; i++) solver.edges[i].dist -= x;
    bool ret = solver.negativeCycle();
    for (int i = 0; i < solver.m; i++) solver.edges[i].dist += x;
    return ret;
}

int main() {
    int T; scanf("%d", &T);
    for (int kase = 1, n, m; scanf("%d%d", &n, &m), kase <= T; kase++) {
        solver.init(n);
        int ub = 0;
        for (int i = 0, u, v, w; i < m; i++)
            scanf("%d%d%d", &u, &v, &w), ub = max(ub, w), solver.AddEdge(u-1, v-1, w);
        printf("Case #%d: ", kase);
        if (!test(ub + 1)) printf("No cycle found.\n"); // 二分之一之前首先确定有解
        else {
            double L = 0, R = ub;
            while (R - L > 1e-3) {
                double M = L + (R - L) / 2;
                if (test(M)) R = M; else L = M;
            }
            printf("%.2lf\n", L);
        }
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.3.2 节例题 15
同类题目：Wormholes, UVa 558
*/

```

例7-18 【差分约束系统】Halum操作 (Halum, UVa 11478)

给定一个 n 个点、 m 条边的有向图 ($n \leq 500$, $m \leq 2700$)，每条边都有一个权值。每次你可以选择一个结点 v 和一个整数 d ，把所有以 v 为终点的边的权值减小 d ，把所有以 v 为起点的边的权值增加 d ($1 \leq u, v \leq n$, $-10\,000 \leq d \leq 10\,000$)，最后要让所有边权的最小值非负且尽量大。输出边权最小值的最大值。如果无法让所有边权都非负，输出“No Solution”；如果边权最小值可以任意大，输出“Infinite”。

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cstring>
#include <queue>
using namespace std;
```

```

const int INF = 1e9, NN = 500 + 10, MM = 2700 + 10;
struct Edge {
    int to, dist;
};

// 邻接表写法
struct BellmanFord {
    int n, m, head[NN], next[MM];
    Edge edges[MM];
    bool inq[NN];           // 是否在队列中
    int d[NN];              // s 到各个点的距离
    int cnt[NN];            // 进队次数

    void init(int n) { this->n = n, m = 0, fill_n(head, n + 1, -1); }
    void AddEdge(int from, int to, int dist) {
        next[m] = head[from], head[from] = m, edges[m++] = (Edge) {to, dist};
    }
    bool negativeCycle() {
        queue<int> Q;
        fill_n(inq, n + 1, false), fill_n(cnt, n + 1, 0);
        for (int i = 0; i < n; i++) d[i] = 0, Q.push(i);
        while (!Q.empty()) {
            int u = Q.front();
            Q.pop(), inq[u] = false;
            for (int i = head[u]; i != -1; i = next[i]) {
                Edge& e = edges[i];
                if (d[e.to] > d[u] + e.dist) {
                    d[e.to] = d[u] + e.dist;
                    if (inq[e.to]) continue;
                    Q.push(e.to), inq[e.to] = true;
                    if (++cnt[e.to] > n) return true;
                }
            }
        }
        return false;
    }
};

```

BellmanFord solver;

// 判断在初始差分约束系统的每个不等式右侧同时减去 x 之后是否有解

```

bool test(int x) {
    for (int i = 0; i < solver.m; i++) solver.edges[i].dist -= x;
    bool ret = solver.negativeCycle();
    for (int i = 0; i < solver.m; i++) solver.edges[i].dist += x;
    return !ret;           // 如果有负环, 说明差分约束系统无解
}

```

```

int main() {
    for (int n, m; scanf("%d%d", &n, &m) == 2;) {
        solver.init(n);
        int ub = 0;
        for (int i = 0, u, v, d; i < m; i++) {
            scanf("%d%d%d", &u, &v, &d);
            ub = max(ub, d);
            solver.AddEdge(u - 1, v - 1, d);
        }
        // 如果可以让每条边权都大于 ub, 说明每条边的权都增加了, 重复一次会增加得更多, 直到无限
        if (test(ub + 1)) puts("Infinite");
        else if (!test(1)) puts("No Solution");
        else {
            int L = 2, R = ub, ans = 1;
            while (L <= R) {
                int M = L + (R - L) / 2;
                if (test(M)) ans = M, L = M + 1;
                else R = M - 1;
            }
            printf("%d\n", ans);
        }
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 5.3.3 节例题 16
同类题目: King, POJ 1364
          Layout, POJ 3169
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点, 如表7-2所示。

表7-2 最短路问题例题归纳